

IF 260

Sistem Operasi

Minggu 8

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

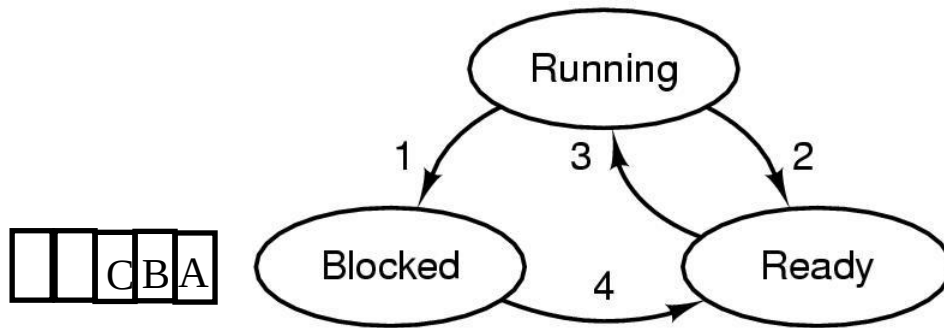
Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Pokok bahasan Minggu 8-14 (Ref: RPKPS IF260)**
- **Topik Minggu 8:**
 - **Deadlock**

Deadlock

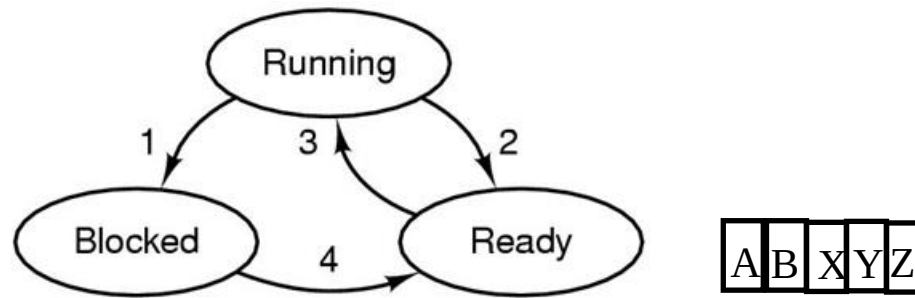
Deadlock



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

- **Diberikan suatu himpunan yang berisikan beberapa process. Dikatakan deadlock apabila tiap process di dalam himpunan ini menunggu event yang hanya bisa dimunculkan oleh process lain di himpunan. Contoh:**
 - **Producer-consumer dgn race condition, producer dan consumer mencapai kondisi sleep dan saling menunggu untuk di-wakeup**
 - **Device deadlock: misal sistem punya 2 tape drives. Ada dua process yang gunakan masing-masing tape drive dan saling menunggu tape drive yang lain**
 - **Apabila hanya ada satu process di himpunan, maka deadlock dapat terjadi pada level thread, dan menjadi tanggung jawab programmer untuk menangani**

Starvation dan Livelock



- **Starvation:** Suatu kondisi di mana process berada di state ready (tidak dalam kondisi blocked), akan tetapi tidak pernah mendapatkan jatah CPU. Ini dapat terjadi karena diberikan prioritas rendah, dan CPU sibuk melayani process-process lain dengan prioritas lebih tinggi.
- **Livelock:** suatu kondisi di mana walau process dapat menggunakan CPU (state running), namun tidak dapat melanjutkan eksekusi sampai selesai. Misalnya apabila sinkronisasi antar process menggunakan busy waiting (spinlock), dan dalam kondisi tiap process saling menunggu lock dilepaskan untuk masuk ke critical region

Potensi Deadlock:

Masalah urutan semaphores

```
semaphore resource_1;  
semaphore resource_2;
```

Contoh:
Inisialisasi = 1

```
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

```
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

Persyaratan Kondisi Deadlock

- **Mutual Exclusion Condition**
 - Pada satu waktu, tiap resource available atau bisa digunakan oleh hanya satu process
- **Hold and Wait Condition**
 - Process yang sedang menggunakan suatu resource masih diperbolehkan untuk request resource lain dan menunggu sampai tersedia
- **No preemption condition**
 - Tidak ada resource yang dapat dilepaskan secara paksa dari process yang sedang menggunakannya → process itu sendiri yang nanti akan melepaskannya
- **Circular wait condition**
 - Kondisi menyerupai lingkaran rantai yang terdiri dari beberapa processes dan resources di mana tiap process menunggu resource yang digunakan oleh process lain di sebelahnya

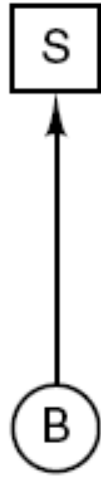
Pemodelan Deadlock dengan Graph Theory (Teori Graf)

- **Graph terdiri atas Nodes dan Arcs**
- **Modelkan deadlock dengan directed graph**
- **Nodes**
 - **Circles (gambar lingkaran) → memodelkan Process**
 - **Squares (gambar kotak) → memodelkan Resource**
- **Directed Arc**
 - **Directed arc dari circle ke square → process request resource dan saat ini sedang blocked menunggu resource tersebut**
 - **Directed arc dari square ke circle → resource sudah di-granted ke process dan process sedang menggunakannya**

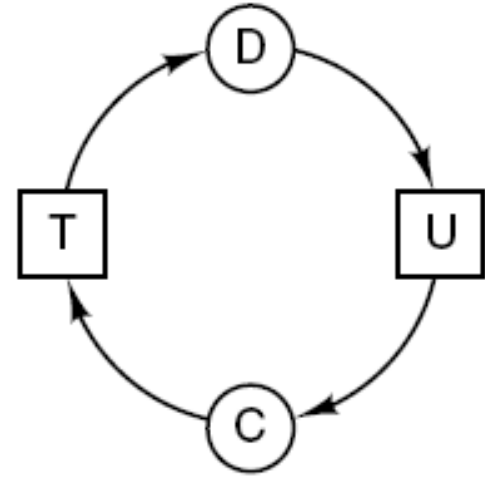
Resource Allocation Graph



(a)



(b)

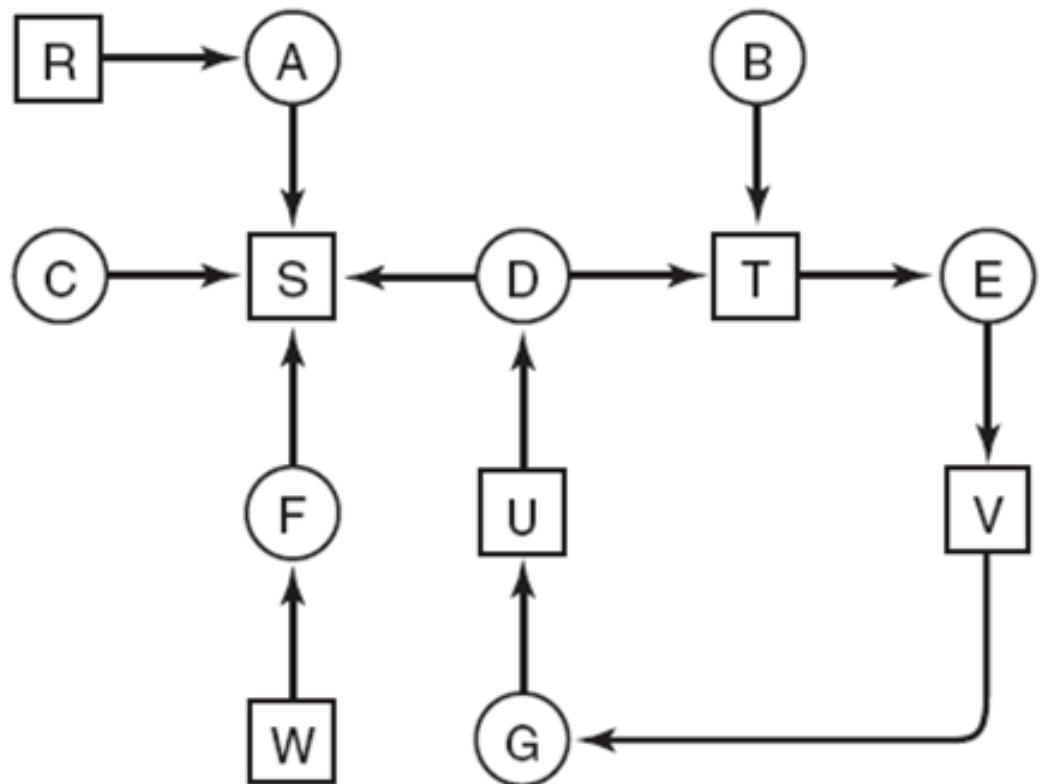


(c)

(a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

Contoh Penggunaan Graph

- Deteksi deadlock dengan satu resource untuk tiap jenis.
- Contoh di bawah: A holds R, want S, B holds nothing, wants T, dst
- Apakah ini deadlock?



Contoh pengaturan alokasi resource menggunakan graph

A
Request R
Request S
Release R
Release S

(a)

B
Request S
Request T
Release S
Release T

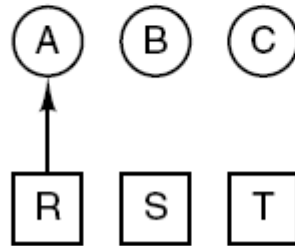
(b)

C
Request T
Request R
Release T
Release R

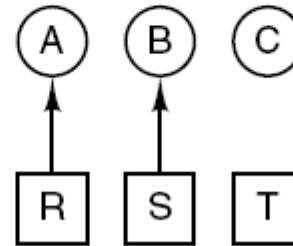
(c)

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R
deadlock

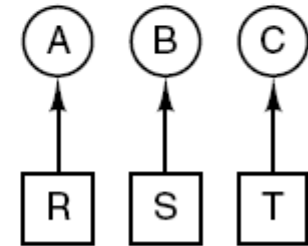
(d)



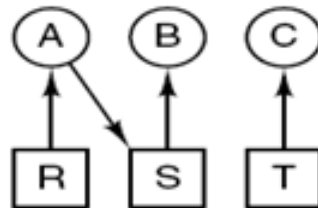
(e)



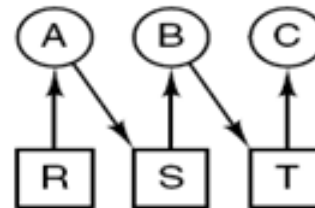
(f)



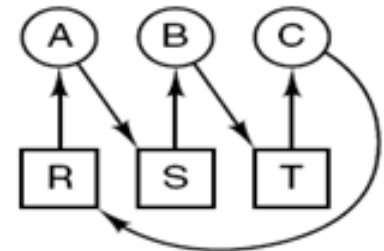
(g)



(h)



(i)



(j)

Bagaimana mengatur pengalokasian untuk menghindari deadlock?

Strategi untuk menghadapi Deadlock

- **Just Ignore the problem → Ostrich Algorithm**
- **Detection and Recovery**
 - Deteksi deadlock kemudian lakukan tindakan
- **Dynamic avoidance**
 - Alokasikan resource secara berhati-hati
- **Prevention**
 - Hilangkan salah satu dari 4 kondisi deadlock

Deadlock Detection & Recovery

- Technique ini membiarkan deadlock terjadi, deteksi terjadinya dan lakukan recovery
- Deadlock detection:
 - One resource of each type
 - Contoh: 1 printer, 1 tape drive, 1 cdrom drive, 1 plotter 1 buah resource lainnya
→ contoh di slide awal tentang adanya cycle di graph sebagai penanda adanya deadlock
 - Multiple resources of each type
 - Contoh: 3 printer, 2 tape drive, 2 cdrom drive, dsb.
 - Gunakan matrix-based algorithm
- Recovery:
 - Rollback, killing process ataupun forced preemption

Deadlock Detection:

Multiple resources of each type

- Notasi Matrix dan Vector
 - **E**: existing resource vector
 - **A**: Available resource vector
 - **C**: Current allocation matrix
 - **R**: Request matrix

Persyaratan:

$$\sum_{i=1}^n C_{ij} + A_j = E_j$$

Resources in existence
($E_1, E_2, E_3, \dots, E_m$)

Resources available
($A_1, A_2, A_3, \dots, A_m$)

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \cdots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \cdots & C_{nm} \end{bmatrix}$$

Row n is current allocation
to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \cdots & R_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \cdots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2 needs

Deadlock Detection:

Multiple resources of each type

- Untuk tiap process P_i , di mana baris ke i dari matrix R kurang atau sama dengan vector $A \rightarrow$ artinya process P_i masih bisa berjalan
- Kalau ada, *mark* (tandai) process P_i , update baris ke i dari C dengan menambahkannya dengan baris ke i dari R , dan update vector A
- Kemudian tambahkan baris ke i dari C ke vector A (artinya process P_i release semua resources setelah selesai)
- Lanjutkan untuk semua process. Apabila ada process yang tidak ditandai (*unmarked*), maka terjadi deadlock
- Contoh:

$$E = \begin{pmatrix} 4 & 2 & 3 & 1 \end{pmatrix}$$

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Request matrix

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Deadlock Detection

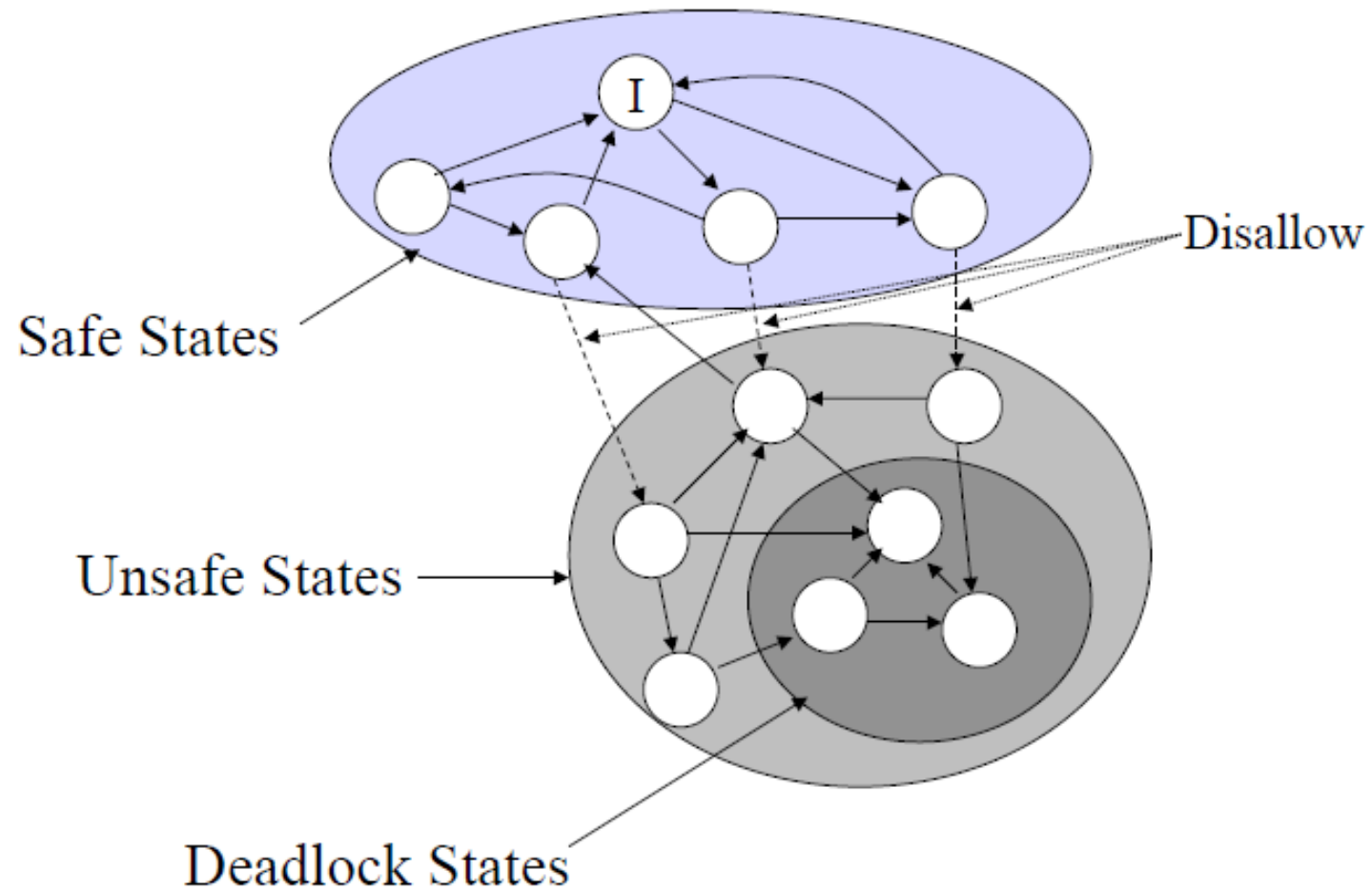
	Request					Allocated					Available				
	R ₀	R ₁	R ₂	R ₃	R ₄	R ₀	R ₁	R ₂	R ₃	R ₄	R ₀	R ₁	R ₂	R ₃	R ₄
P ₀	0	1	0	0	1	1	0	1	1	0	0	0	0	0	1
P ₁	0	0	1	0	1	1	1	0	0	0					
P ₂	0	0	0	0	1	0	0	0	1	0					
P ₃	1	0	1	0	1	0	0	0	0	0					

- Tandai P₃ karena belum ada *resources* yang dialokasikan
- Tandai P₂ karena P₂'request ≤ vector Available, kemudian tentukan:
 - Vector Available = Available + (0,0,0,1,0) = (0,0,0,1,1)
- Algorithm terminates → Deadlock

Deadlock Avoidance

- Slide sebelumnya mengambil asumsi process mengambil resource yang di-request secara bersamaan.
- Kalau diambil satu persatu, apakah dapat dirancang algoritma yang menghindari deadlock (deadlock avoidance).
- Konsep safe dan unsafe state
 - Safe state: state di mana terdapat suatu scheduling order (urutan penjadwalan) sehingga semua process dapat berjalan sampai selesai. (bahkan walaupun process request semua resources yang diperlukan secara serentak)
 - Unsafe state: tidak dapat menjamin semua process untuk selesai. Unsafe state tidak berarti deadlock, karena masih dapat berjalan sebelum nantinya terkena deadlock
- Banker's algorithm (lihat Tanenbaum hal 449-452)
 - Tiap process deklarasikan jumlah maksimum resource yang diinginkan
 - Setiap terjadi perubahan state karena ada request yang dipenuhi, state tersebut haruslah safe state

Safe, Unsafe, Deadlock States



Safe & Unsafe State:

3 processes and 1 jenis resource

Total resource = 10

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	3	9
B	4	4
C	2	7

Free: 1
(b)

	Has	Max
A	3	9
B	0	—
C	2	7

Free: 5
(c)

	Has	Max
A	3	9
B	0	—
C	7	7

Free: 0
(d)

	Has	Max
A	3	9
B	0	—
C	0	—

Free: 7
(e)

(a) adalah safe state

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	4	9
B	2	4
C	2	7

Free: 2
(b)

	Has	Max
A	4	9
B	4	4
C	2	7

Free: 0
(c)

	Has	Max
A	4	9
B	—	—
C	2	7

Free: 4
(d)

(b) adalah unsafe state

Contoh Soal

Terdapat 4 processes dan 5 resources yang akan dialokasikan. Alokasi resources saat ini (Allocated), maksimum alokasi resources yang diinginkan (Maximum) dan resources yang masih tersisa (Available) diberikan pada tabel di bawah.

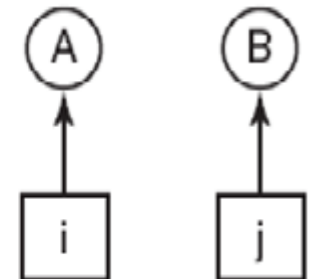
Berapa nilai minimum x sehingga kondisi ini tergolong safe state?

	Allocated					Maximum					Available				
Process A	1	0	2	1	1	1	1	2	1	3	0	0	x	1	2
Process B	2	0	1	1	0	2	2	2	1	0	R1	R2	R3	R4	R5
Process C	1	1	0	1	0	2	1	3	1	0					
Process D	1	1	1	1	0	1	1	2	2	1					
	R1	R2	R3	R4	R5	R1	R2	R3	R4	R5					

Deadlock Prevention

- **Attacking the mutual exclusion condition**
 - Rancang hanya satu process saja yang akan mengakses resource. Contoh: printer daemon dan spooling.
- **Attacking the hold and wait condition**
 - Request seluruh resources yang diperlukan sebelum eksekusi
 - Jika process request resource baru, lepaskan semua yang di-hold, dan kemudian dapatkan seluruh resources yang diperlukan secara serentak
- **Attacking the no-preemption condition**
- **Attacking the circular wait condition**
 - Hindari terbentuknya cyclic graph
 - Penomoran resources secara global, tiap process akses resources sesuai urutan nomor

1. Imagesetter
2. Scanner
3. Plotter
4. Tape drive
5. CD-ROM drive



Contoh Soal

Sebuah sistem komputer memiliki sumber daya sbb.: 7 tape drive, 5 plotter, 4 scanner, dan 4 cdrom. Terdapat 4 buah process P_1 , P_2 , P_3 , dan P_4 yang sedang berjalan, di mana:

- P_1 : sedang menggunakan 2 tape drive, 1 plotter dan 1 cdrom, dan masih memerlukan 1 tape drive, 1 plotter dan 1 scanner.
 - P_2 : sedang menggunakan 1 tape drive, 2 plotter dan 1 scanner, dan masih memerlukan 1 tape drive, dan 1 cdrom.
 - P_3 : sedang menggunakan 2 tape drive, 1 scanner, dan 1 cdrom, dan masih memerlukan 2 plotter, 1 scanner dan 2 cdrom.
 - P_4 : sedang menggunakan 1 plotter, 2 scanner dan 1 CDROM, dan masih memerlukan 2 tape drive, 1 plotter dan 3 cdrom.
- a. Buat matrix-matrix yang berisikan sumber daya teralokasi (**Allocated**), maksimum alokasi sumber daya yang diinginkan (**Maximum**), dan sumber daya yang masih tersisa (**Available**).
- b. Lakukan analisa untuk mengetahui apakah akan terjadi deadlock ?

Diberikan 6 buah process yaitu P0, P1, P2, P3, P4, P5, dan 4 tipe resource, yaitu tipe A (sejumlah 15), tipe B (sejumlah 6), tipe C (sejumlah 9), dan tipe D (sejumlah 10). Saat T0, status penggunaan resource adalah sebagaimana di bawah:

- Tunjukkan apakah status saat ini tergolong safe atau unsafe state? Jelaskan!
- Jika process P5 request (3,2,3,3), jelaskan apakah request ini sebaiknya dipenuhi atau tidak? Jelaskan!

Available								

Akhir Kuliah Minggu 8

Terima kasih atas perhatiannya!